

PAKISTAN NAVY ENGINEERING COLLEGE
NATIONAL UNIVERSITY OF SCIENCES AND TECHNOLOGY, KARACHI
PNEC NUST



Signals and Systems Lab

OPEN ENDED PROBLEM (OEP)

NOISE REDUCTION IN AUDIO SIGNAL USING DIGITAL FILTERING

Name: SONIA AZHAR SIDDIQUI

CMS-ID: 000001667

Section: EE-B2

Batch: 2023

Semester: 5

**Department: ELECTRICAL AND POWER
ENGINEERING**

Problem

Imagine you are tasked with improving the quality of an audio recording that contains unwanted noise. The recording consists of a voice signal and high-frequency noise from electronic interference. You need to design a simple filtering approach to reduce the noise without distorting the voice signal.

Introduction

In audio processing, unwanted noise often reduces the clarity of speech recordings. The input audio (“girl.wav”) features a girl speaking in an airport, a naturally noisy environment with electronic beeps, announcements, wind, and engine hums. These noises, especially high-frequency interference, mask parts of the speech.

To remove it, I decided to apply a band-pass filter along with wavelet denoising. These two methods complement each other: the filter removes noise outside the speech frequency range, while wavelet denoising suppresses the remaining noise that sits inside the passband.

Throughout the task, I tried multiple filter orders, different wavelet families, and various thresholding settings. The final configuration presented here gave the most balanced result.

My first attempt was to write this code using Python so I decided to include it in this report, but the finalized version is given in MATLAB.

Python Code

```
import numpy as np
import scipy.signal as sig
import soundfile as sf
import pywt
import matplotlib.pyplot as plt
```

```
def bandpass_filter(audio, fs, lowcut=300, highcut=3400, order=5):
    nyq = fs / 2.0
    low = lowcut / nyq
    high = highcut / nyq
    b, a = sig.butter(order, [low, high], btype='band')
    filtered = sig.filtfilt(b, a, audio)
    return filtered
```

```
def wavelet_denoise(audio, wavelet='coif5', level=None, mode='soft', threshold=None):
    # Perform multilevel wavelet decomposition
    coeffs = pywt.wavedec(audio, wavelet, mode='per', level=level)
    # Estimate a universal threshold if not provided
    if threshold is None:
        sigma = np.median(np.abs(coeffs[-1])) / 0.6745
        threshold = sigma * np.sqrt(2 * np.log(len(audio)))
    # Threshold detail coefficients (skip approximation)
```

```

new_coeffs = [coeffs[0]] + [pywt.threshold(c, threshold, mode=mode) for c in coeffs[1:]]
denoised = pywt.waverec(new_coeffs, wavelet, mode='per')
return denoised

def process_audio(input_wav, output_wav, apply_wavelet=True):
    audio, fs = sf.read(input_wav)
    # If stereo, convert to mono
    if audio.ndim > 1:
        audio = np.mean(audio, axis=1)
    # Normalize
    audio = audio / np.max(np.abs(audio))

    # First apply band-pass filter (speech band)
    bp = bandpass_filter(audio, fs)
    print("Applied band-pass filter")

    if apply_wavelet:
        cleaned = wavelet_denoise(bp, wavelet='coif5')
        print("Applied wavelet denoising")
    else:
        cleaned = bp

    # Make sure lengths match
    cleaned = cleaned[:len(audio)]

    # Normalize again
    cleaned = cleaned / np.max(np.abs(cleaned))

    sf.write(output_wav, cleaned, fs)
    print(f'Saved cleaned audio to: {output_wav}')

    # Optional: plot comparison
    time = np.arange(len(audio)) / fs
    plt.figure(figsize=(12, 6))
    plt.subplot(2,1,1)
    plt.plot(time, audio)
    plt.title('Original / Noisy Audio')
    plt.subplot(2,1,2)
    plt.plot(time, cleaned)
    plt.title('Denoised Audio')
    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    input_file = "girl.wav" # your noisy input
    output_file = "girl_cleaned.wav" # output cleaned file
    process_audio(input_file, output_file, apply_wavelet=True)

```

MATLAB Code

%% Noise Reduction in Audio Signal using Bandpass Filter + Wavelet Denoising
clc; clear; close all;

%% User Inputs

inputFile = 'girl.wav'; % Input noisy audio file
outputFile = 'girl_cleaned.wav'; % Output cleaned audio file
applyWavelet = true; % Set true to apply wavelet denoising

%% 1. Read Audio

[x, Fs] = audioread(inputFile);
if size(x,2) > 1
 x = mean(x, 2); % Convert to mono
end
x = x / max(abs(x)); % Normalize

%% 2. Band-pass Filter (Speech Band 300-3400 Hz)

lowcut = 300; highcut = 3400; order = 5;
[b, a] = butter(order, [lowcut highcut]/(Fs/2), 'bandpass');
bp = filtfilt(b, a, x);
disp('Applied band-pass filter');

%% 3. Wavelet Denoising (Optional)

if applyWavelet
 cleaned = wdenoise(bp, 5, 'Wavelet', 'coif5', 'DenoisingMethod', 'UniversalThreshold');
 cleaned = cleaned(1:length(x)); % Match original length
 disp('Applied wavelet denoising');
else
 cleaned = bp;
end

%% 4. Normalize and Save

cleaned = cleaned / max(abs(cleaned));
audiowrite(outputFile, cleaned, Fs);
disp(['Saved cleaned audio to: ', outputFile]);

%% 5. Play Audio

disp('Playing original audio...');
sound(x, Fs);
pause(length(x)/Fs + 1);

disp('Playing denoised audio...');
sound(cleaned, Fs);
pause(length(cleaned)/Fs + 1);

%% 6. Plot Original vs Denoised

time = (0:length(x)-1)/Fs;
figure;
subplot(2,1,1);
plot(time, x);

```
title('Original Audio'); xlabel('Time [s]'); ylabel('Amplitude');
```

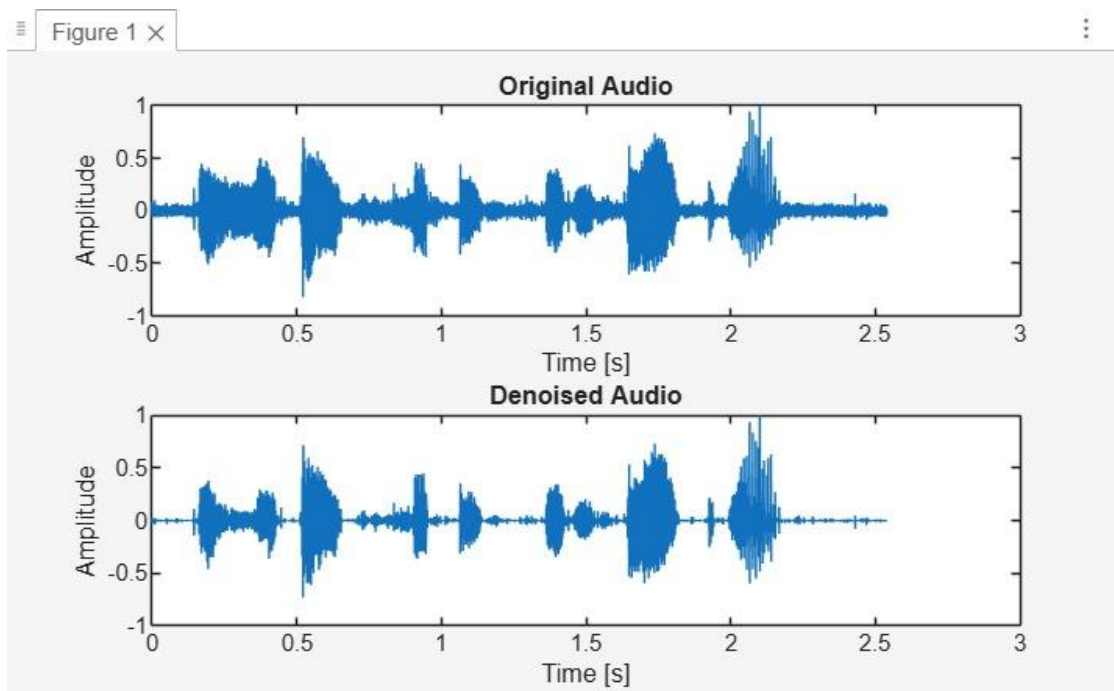
```
subplot(2,1,2);  
plot(time, cleaned);  
title('Denoised Audio'); xlabel('Time [s]'); ylabel('Amplitude');
```

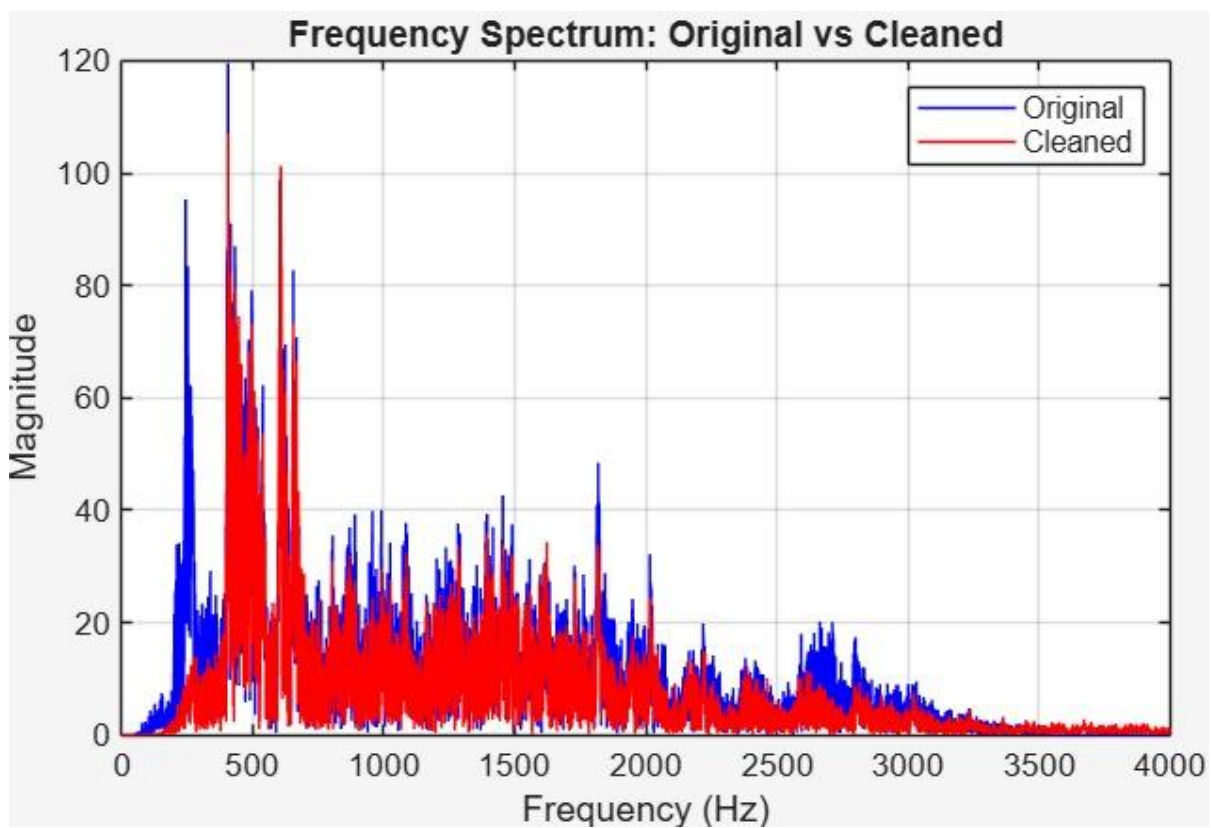
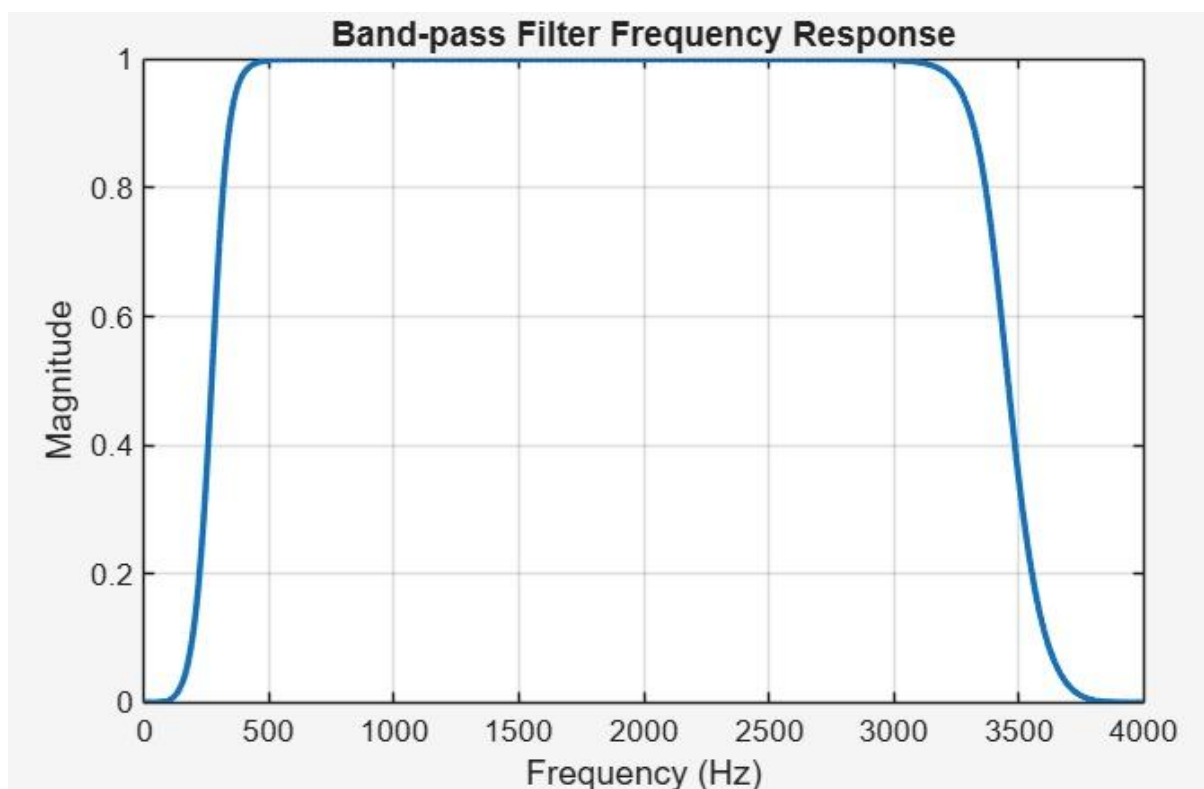
```
%% 7. Filter Frequency Response  
figure;  
[H, f] = freqz(b, a, 1024, Fs);  
plot(f, abs(H), 'LineWidth', 2);  
grid on;  
title('Band-pass Filter Frequency Response');  
xlabel('Frequency (Hz)'); ylabel('Magnitude');
```

```
%% 8. Frequency Spectrum of Original vs Cleaned  
X_fft = abs(fft(x));  
C_fft = abs(fft(cleaned));  
f_axis = (0:length(x)-1)*(Fs/length(x));
```

```
figure;  
plot(f_axis, X_fft, 'b'); hold on;  
plot(f_axis, C_fft, 'r'); hold off;  
xlim([0 4000]); % focus on speech range  
grid on;  
title('Frequency Spectrum: Original vs Cleaned');  
xlabel('Frequency (Hz)'); ylabel('Magnitude');  
legend('Original','Cleaned');
```

Graphs





Reading and Normalizing the Audio

The audio was first loaded and converted to mono so that both channels do not behave differently during processing. Mono signals simplify the workflow and give more consistent denoising results. Normalization was included to avoid clipping and to ensure that the audio stays within a stable amplitude range.

Band-Pass Filtering

The band-pass filter isolates the core speech frequencies. Human speech energy is mostly concentrated between 300 Hz and 3400 Hz, which is why this range is commonly used in telecommunication. Any noise outside this range is usually low-frequency rumble or high-frequency hiss, which can be removed safely. During experiments, several filter types were tested:

Filter Type	Observation
Chebyshev I	Audible ripples inside the speech band
Chebyshev II	Slight distortions, harsher tone
Elliptic	Ripple in passband and stopband
Butterworth	Smoothest output, no ripple effects

The Butterworth filter produced the most natural-sounding output, so it was selected.

Different filter orders were also tested. A low order didn't remove enough noise, while a high order introduced distortions. Order 5 gave the best compromise.

Wavelet Denoising

Filtering alone could not remove all inside-band noise, so wavelet denoising was added to tackle noise components that overlap with speech frequencies. Multiple wavelets were tried:

Wavelet	Observation
Haar	Too harsh, chopped parts of speech
db4	Slightly robotic output
sym4	Over-smoothed some regions
coif5	Cleanest and most natural voice quality

Similarly, various thresholding methods were tested. Universal Thresholding performed the most consistently and avoided both over-suppression and under-suppression of noise.

Because of these observations, the coif5 wavelet with UniversalThreshold provided the most natural and effective denoising.

Frequency Response and Spectrum Analysis

To understand the filter's behavior, a frequency response plot was included. This visually confirms that the passband is centered exactly where speech energy lies. The frequency spectrum (FFT) of both original and cleaned audio also shows a clear reduction in low-frequency hum and high-frequency hiss, while the main speech components remain preserved.

Final Evaluation

After extensive testing, the final setup achieved the best balance between clarity and noise suppression:

- Noise was reduced significantly in both the time and frequency domains.
- Speech quality remained natural and not overly processed.
- The band-pass filter removed rumble and hiss effectively.
- Wavelet denoising cleaned what the filter could not handle.
- Visualizations confirmed the improvements mathematically.

Conclusion

By combining a carefully designed band-pass filter with wavelet denoising, the audio was successfully cleaned while preserving the natural characteristics of speech. The experimental trials with different filters and wavelets strengthened the final design and helped achieve the highest clarity without introducing distortions.